

Trabajo Obligatorio N°1

Implementación del juego Truco con Prolog



Alumnos

Ferraris, Facundo
Pereira, Marisol
Andino, Joaquin
Carrasco, Daniel

Docentes

Vaucheret Claudio
Coronel Paula

Índice

Introducción	3
Consideraciones adoptadas	4
Estructura de Archivos	4
Mazo	4
Ronda	6
Jugador	6
Funcionalidad principal del programa	7
Mezcla y reparto de cartas	8
Turno de un jugador	8
Lógica de rondas y manos	8
Envido	9
Truco	10
Flujo principal	11

Introducción

El presente informe se llevó a cabo con el fin de reflejar los conocimientos aprendidos en la materia de Lenguajes Declarativos. Para ello, se propuso la realización de un juego de cartas con prolog, aplicando gramáticas de cláusulas definidas (DCG) para la máquina de estados, y notaciones semi contextuales para manipular los estados globales de la partida.

El grupo acordó que el juego será el truco, que cuenta con reglas complejas, decisiones estratégicas y numerosos estados posibles durante la partida, considerándose conveniente para el modelado utilizando relaciones lógicas y transformaciones de estado. El programa fue realizado y probado con **SWI-Prolog 9.2.9**, de la mano con librerías tales como random y clpfd, para simular barajar el mazo y para las comparaciones numéricas en la jerarquía de cartas respectivamente.

A continuación se explicarán con mayor detalle, las consideraciones e implementaciones de cada funcionalidad del programa.

Consideraciones adoptadas

Para mantener el trabajo dentro de un alcance razonable, tomamos las siguientes decisiones de diseño:

- **Modalidad 1 vs 1:** la partida es entre dos jugadores únicamente, identificados como *jugador1* y *jugador2*. Esto nos permitió concentrarnos en la lógica del juego sin agregar la complejidad de equipos y mazo compartido.
- **Sin Flor:** decidimos no incluir la Flor porque requiere detectar tres cartas del mismo palo antes de la primera mano y agrega un subsistema de negociación completo que excede el foco del TP.
- **Partida a 15 puntos:** usamos la modalidad corta, que es la más común en partidas de a dos jugadores.

Estructura de Archivos

Los archivos del juego fueron organizados por módulos para evitar la sobrecarga de lógica en un solo archivo.

Truco	
└ config.pl	→ Configuraciones base del juego
└ gestor_estado.pl	→ Estados del juego
└ interfaz.pl	→ Entradas de teclado, y selección de opciones
└ mazoTruco.pl	→ Cartas, jerarquías, y cálculo de puntuaciones
└ motor_juego.pl	→ Núcleo del juego, donde conecta todos los módulos
└ sistema_cantos.pl	→ Contiene la lógica de los cantos
└ truco.pl	→ Archivo main donde se ejecuta el juego

Estructura del programa

Toda la partida fue modelada mediante un estado global que contiene la información necesaria en cada momento del juego. La estructura general del estado es:

```
[mazo(...), Jugadores(...), Ronda(...)]
```

Mazo

```
mazo(Cartas) .
```

Utilizamos el mazo español de 40 cartas, en donde representamos a cada carta como Palo-Valor. Abreviamos los palos como *c* para copa, *o* para oro, *b* para basto, y *e* para espada

Por ejemplo:

- e-1. (1 de espada)
- o-7. (7 de oro)
- b-3. (3 de bastos)

La jerarquía entre las cartas la representamos asignándole un valor numérico a cada carta, si el valor numérico de una carta es mayor a otro valor de otra carta quiere decir que es de mayor jerarquía. En nuestro ejemplo anterior, el valor numérico de e-1 es 14, mientras que el valor numérico de o-7 es 11, y el de b-3 es 10.

La jerarquía completa de cartas es:

```
e-1 (14)
b-1 (13)
e-7 (12)
o-7 (11)
_-3 (10)
_-2 (9)
c-1 (8)
o-1 (8)
_-12 (7)
_-11 (6)
_-10 (5)
c-7 (4)
b-7 (4)
_-6 (3)
_-5 (2)
_-4 (1)
```

Ronda

```
ronda(Resultados, CantoActual, Rechazo, EstadoEnvido).
```

Resultados: En este parámetro guardamos el historial de la ronda, es decir qué jugador ganó. Por ejemplo:

```
[] //Aún no ganó nadie.  
  
[jugador(jugador1,_,_)] //El jugador 1 ganó una mano.  
  
[parda, jugador(jugador2,_,_)] //Hubo un empate y el jugador 2 ganó una mano.
```

Canto Actual: Indica el nivel del truco en el que está la mano actual. Los cantos posibles son:

```
es_canto(truco).  
es_canto(retruco).  
es_canto(vale4).
```

Rechazo: Indica si algún canto de truco fue rechazado, un ejemplo de lo que puede contener sería:

```
rechazo(jugador1)
```

Estado envido: Este parámetro lo utilizamos para almacenar si el envido no fue cantado, fue aceptado o ya fue rechazado.

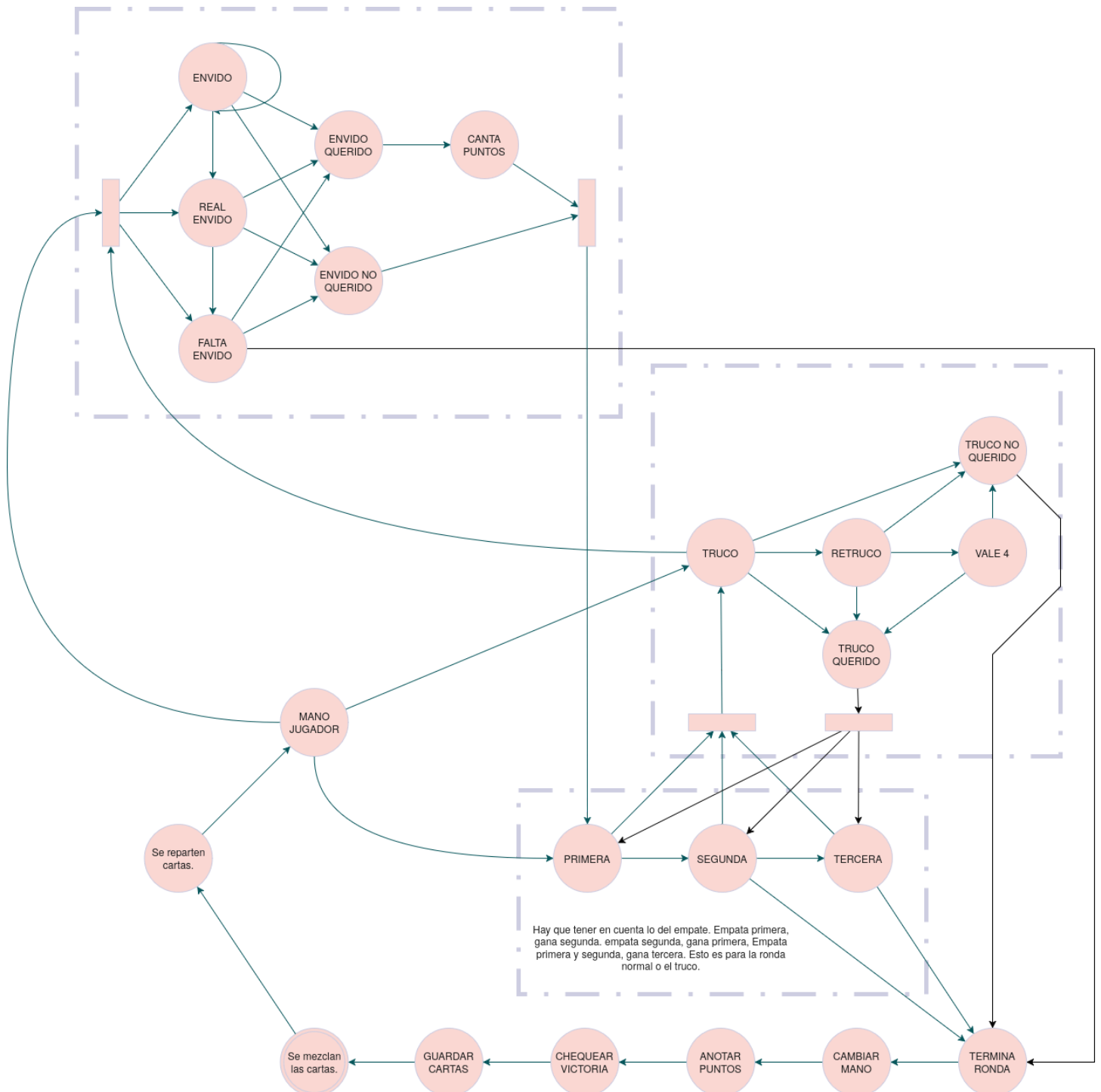
Jugador

```
jugador(NombreJugador, Mano, Puntaje).
```

En *jugador* guardamos el nombre del jugador, las cartas que tiene en la mano actual, y el puntaje actual de la ronda.

Funcionalidad principal del programa

En el siguiente diagrama podemos visualizar las funciones principales que fuimos implementando, teniendo como estado inicial “Se mezclan cartas” y el estado final “Termina ronda”.



Mezcla y reparto de cartas

Al iniciar el juego, simulamos el mezclado de cartas las cartas con el siguiente predicado:

```
mezclar(Cartas, CartasMezcladas)
```

En donde ingresamos el mazo de cartas, y devolvemos el mazo de cartas mezclado. Luego, repartimos las cartas con el siguiente predicado:

```
repartir_carta_a_cada_jugador(+JugadoresAntes, -JugadoresDespues, +MazoAntes, -MazoDespues)
```

En donde repartimos una carta a cada jugador, y cada carta que repartimos la quitamos del mazo.

Turno de un jugador

Durante su turno, un jugador puede hacer dos cosas: jugar una carta o cantar envideo o truco. El predicado

```
turno_jugador(+Nombre, -CartaJugada, -TerminaRonda)
```

Permite las siguientes acciones:

1. Jugar una carta de su mano;
2. Cantar truco o envideo;
3. Ingresar una acción inválida.

Si elige jugar, se le pide que elija una carta. Si elige cantar, se le pide el canto y se deriva a **resolver_canto_o_envideo_en_turno**, que decide si es un canto de envideo o de truco y lo procesa con el subsistema correspondiente.

Lógica de rondas y manos

Una ronda se compone de hasta tres manos. Cada mano enfrenta una carta de cada jugador y determina quién gana o si hay parda.

La ronda termina en cualquiera de estos tres casos:

- Hubo rechazo de un canto de truco.
- Se jugaron tres manos.
- Algún jugador ganó dos manos.

Esto lo verificamos con:

```
ronda_terminada(+Resultados, +Rechazo)
```


En donde en *Resultados* tenemos la lista de resultados de las manos jugadas hasta el momento, y en *Rechazo* el estado del rechazo de la ronda.

La ronda la finalizamos con el estado:

```
finalizar_ronda
```

El cuál realiza las siguientes acciones:

1. Obtiene el ganador con ***obtener_ganador/3***;
2. Calcula cuántos puntos vale la ronda según el canto vigente;
3. Actualiza el puntaje del ganador;
4. Reinicia la estructura ronda(...) con ***estado_envido_inicial/1***.

Envido

El envido es posible cantarlo cuándo:

- El juego actual está en primera mano.
- Nadie se fue al mazo.
- Aún no fue cantado.

En el envido, le asignamos a cada carta un valor numérico para calcular el puntaje de este con el siguiente predicado:

```
valor_envido_numero(+Numero, -Valor)
```

Tuvimos en cuenta las posibles cadenas de envido válidas que pueden surgir en el juego, esto lo definimos en el siguiente predicado:

```
canto_envido_valido(cantoActual, cantoValido).
```

Por ejemplo:

```
canto_envido_valido([envido,envido,real_envido],falta_envido).
```

Si se canta envido, luego envido, luego real envido, el único canto que queda y que es válido en esta secuencia es falta envido.

El cálculo del puntaje del envido de una mano sigue estas reglas:

- Si hay dos o más cartas del mismo palo, el valor es 20 más la suma de los dos mejores valores de ese palo.
- Si no hay par del mismo palo, el valor es el mayor valor individual de las tres cartas.
- Las figuras (10, 11, 12) valen 0 en el envido.
- El as vale 1.

El predicado **envido_habilitado** verifica que la lista de resultados de manos está vacía y que el subestado del envideo sea **no_cantado**.

Truco

El subsistema de truco maneja la cadena de cantos *truco-retruco-vale4*, la negociación entre los dos jugadores, y la asignación de puntos según el resultado.

Los puntos están definidos como:

- Truco aceptado *vale 2, retruco 3, vale4 4*.
- Si se rechaza el *truco* gana 1, *retruco 2, vale4 3*.

La validación de si un canto es legal se hace en:

```
puede_cantar_estado(+NuevoCanto)
```

Que lee el estado actual y verifica tres condiciones: que no haya un rechazo vigente en la ronda, que el término ingresado sea un canto reconocido, y que ese canto supere al canto actual. La comparación se hace con:

```
canto_supera(Nuevo, CantoActual)
```

Este traduce cada canto a un nivel numérico (ninguno=0, truco=1, retruco=2, vale4=3) y compara.

Cuando un jugador canta, el rival puede aceptar, rechazar, o resubir. En caso de resubir, los roles se invierten y se vuelve a pedir respuesta. Esto se modela con recursión en:

```
resolver_respuesta_canto(+Cantor, +Rival, +Canto, +Respuesta)
```

Que intercambia *Cantor* y *Rival* en cada llamada recursiva, simulando así una interacción entre el jugador que canta y el jugador que acepta o rechaza. Si la respuesta es inválida, se toma como rechazo por defecto.

Flujo principal

El bucle de la partida se implementa con recursión DCG en ***jugar_truco***. Tiene dos cláusulas: la primera verifica si la partida terminó con ***fin_partida*** y termina;

La segunda cláusula realiza lo siguiente:

1. Toma los jugadores actuales;
2. Reinicia sus manos con ***nueva_mesa/2***;
3. Cambia quien es mano con ***cambiar_mano/2***;
4. Reinicia la ronda y el envideo;
5. Reparte tres cartas a cada jugador;
6. Juega la mesa completa;
7. Vuelve recursivamente al inicio del ciclo.

En ***jugar_mesa*** a su vez llama a ***jugar_mano*** repetidamente hasta que ***ronda_terminada*** detecte el cierre. En cada ciclo verifica también si ***hay_ganador_partida_estado*** para detener el juego inmediatamente si alguno llegó a 15 puntos durante el envideo, sin esperar a que termine la ronda. El punto de entrada para ejecutar el juego es simplemente: ***?- truco***.

Entradas por teclado

Para las acciones ingresadas por teclado se realizó un predicado de modo que dada una lista, lo que se ingrese debe ser miembro de ésta, en caso contrario, se debe repetir el proceso hasta que sea verdadero. Para prevenir errores que puedan cortar la ejecución, se realizó un catch que toma el error y cambia la palabra por ***error***. Se utilizó el predicado ***read_line_to_String/2***, que consiste en transformar en String lo que llega del primer argumento (para tomar el input del teclado, se utiliza la flag ***user_input***), y luego un ***term_string/1*** para obtener la versión argumento del string.

Conclusión

Se logró implementar una versión funcional del Truco argentino en SWI-Prolog en donde el estado del juego se modela como un término único que se transforma mediante DCG, y la notación semi contextual permite encadenar transformaciones sin nombrar manualmente cada versión intermedia del estado.

El paradigma lógico resultó particularmente adecuado para este problema. Las reglas del Truco, la jerarquía de cartas, las secuencias válidas de cantos de envideo, las condiciones de habilitación de cada acción, se expresan de forma natural como hechos y cláusulas sin necesidad de estructuras de control imperativas. Lo que en otro lenguaje requeriría una serie de ifs anidados, acá se describe directamente como una relación.

La principal dificultad fue el manejo correcto de la cinta DCG. Entender exactamente cuando una variable de lectura puede interferir con una variable de escritura posterior, y por que eso causa un fallo silencioso en lugar de un error visible, requirió analizar el problema con cuidado. Una vez identificado el patrón, la corrección fue simple, pero encontrarlo llevo tiempo.

En perspectiva, el trabajo mostró que Prolog no es solo un lenguaje para resolver puzzles lógicos o procesar listas. Con la técnica adecuada, en este caso DCG como máquina de estados, puede usarse para modelar sistemas dinámicos complejos con un código que es declarativo, modular y relativamente fácil de razonar.

Bibliografía

[1] [SWI-Prolog Reference Manual](#).